

Weekly Documentation Assignment By: Fredricka David

During Week 2, I learned more about Linux, open-source software, the command line, and how to find help when working in a Linux environment. The topics helped me understand not only how Linux works, but also why the Linux command line is important for system administration, cloud engineering, and automation.

One of the main things I learned was the difference between **source code, compiled software, and interpreted software**. Source code is written in a human-readable programming language. With compiled languages, a compiler converts the source code into machine-readable instructions that can be executed by a computer. Interpreted languages, such as Python and Bash, are processed by an interpreter when they are run.

I also learned the difference between **closed-source and open-source software**. With closed-source software, users normally receive the compiled application without access to the source code. The software is controlled by the owner through licensing agreements, which can restrict activities such as copying, modifying, or reverse engineering the software.

Open-source software makes the source code available to users. Depending on the license, users may be able to study, modify, improve, and redistribute the software. This is important because it allows developers and users to inspect how software works and contribute improvements. Open-source development also encourages collaboration and shared responsibility for software quality and security.

I learned that **Linux developed alongside the open-source philosophy** and is based on many concepts associated with UNIX. Linux was written primarily in C and has grown through contributions from developers and organizations around the world. This community-based development has helped Linux become widely used because it can be adapted to many different environments.

Another important concept I learned was **software licensing**. A license defines what users are allowed to do with software. The main areas to consider are ownership, money or payment, and licensing rights. I learned about FOSS, which stands for Free and Open Source Software. In this context, “free” refers mainly to freedom rather than simply meaning that software has no cost.

I also learned about organizations such as the **Free Software Foundation (FSF)** and the **Open Source Initiative (OSI)**. The FSF focuses on software freedom and promotes licenses such as the GNU General Public License (GPL). The GPL is a copyleft license, meaning that certain modified and redistributed versions must remain under the same licensing principles. I also learned that licenses such as BSD and MIT are more permissive and allow software to be used in different types of projects, including closed-source products, subject to their license conditions.

Another area I learned about was the importance of **standards and interoperability**. Standards such as POSIX help different systems follow common approaches, making software more portable and allowing different systems and applications to work together more easily.

A major part of the week focused on the **Linux command line**. I learned that the command line provides direct control over a Linux system and can be faster and more flexible than using a graphical interface. It is especially useful for system administration, automation, troubleshooting, and working with remote servers.

The **shell** is the program that interprets commands entered in the terminal. Bash is one of the most commonly used shells on Linux. I learned that the shell can also provide useful features such as command history, command-line editing, aliases, variables, and scripting.

I learned how Linux commands are normally structured:

```
command [options] [arguments]
```

The command tells Linux what action to perform, options modify how the command behaves, and arguments provide information such as files or directories to work with. For example, `ls -l` uses the `-l` option to provide a detailed listing. Options can also be combined, such as `ls -ltr`.

Linux is **case-sensitive**, which means that uppercase and lowercase letters are treated differently. I also learned how to use command history. The Up Arrow can be used to access previous commands, while the `history` command displays previously executed commands. Commands such as `!!` can repeat the previous command, while `!ls` can execute the most recent command beginning with `ls`.

I also learned about **shell variables**. A local variable exists only within the current shell session and is normally lost when the shell is closed. Variables can be created using a format such as `variable=value` and displayed using `echo $variable`.

Another important skill I developed was learning how to **find help in Linux**. Linux provides built-in documentation through manual pages, commonly called **man pages**. I can use `man` command, for example `man ls`, to view information about a command. The manual page normally contains sections such as the command name, usage or syntax, description, options, related files, author information, and other references.

I learned how to understand common symbols in the SYNOPSIS section of a man page. Square brackets `[ ]` normally indicate optional information, `...` indicates repetition, and `|` represents alternatives. This makes it easier to understand how a command should be used without having to memorise every command.

Linux man pages are organised into different sections covering areas such as general commands, system calls, library calls, file formats, system administration commands, and kernel routines. This is useful when the same name exists in more than one section. For example, `man 5 passwd` can be used to specifically view information about the `passwd` file format.

I also learned different ways to search for information. `man -f` or `whatis` can provide a short description of a command. `man -k` or `apropos` can search for commands based on keywords. The `whereis` command can locate a command, its source files, and its manual pages.

Finally, I learned about **locate**, which can be used to search for files and directories using a database. Commands such as `locate -c` can count results, while `locate -b` can limit the search to filenames. Because `locate` uses a database, newly created files may not immediately appear until the database is updated.

Overall, Week 2 helped me build a stronger foundation in Linux. I learned how open-source software and licensing work, how to use the Linux command line and shell, how commands, options, arguments, variables, and history work, and how to find information using Linux documentation and search tools. These skills are important for working with Linux servers, cloud environments, troubleshooting, automation, and system administration.

1. Get help on a command

>_ Ubuntu PC ▾

```
LS(1)                                User Commands                                LS(1)

NAME
    ls - list directory contents

SYNOPSIS
    ls [OPTION]... [FILE]...

DESCRIPTION
    List information about the FILES (the current directory by default).
    Sort entries alphabetically if none of -cftuvSUX nor --sort is speci-
    fied.

    Mandatory arguments to long options are mandatory for short options
    too.

    -a, --all
        do not ignore entries starting with .

    -A, --almost-all
        do not list implied . and ..

    --author

Manual page ls(1) line 1 (press h for help or q to quit)
```

2. `ls -lh` shows the contents of your current directory (your home directory) in a detailed, human-readable format.

>_ Ubuntu PC ▾

```
sysadmin@localhost:~$ ls -lh
total 4.0K
drwxr-xr-x 2 sysadmin sysadmin  6 Feb  8 2021 Desktop
drwxr-xr-x 4 sysadmin sysadmin 4.0K Feb  8 2021 Documents
drwxr-xr-x 2 sysadmin sysadmin  6 Feb  8 2021 Downloads
drwxr-xr-x 2 sysadmin sysadmin  6 Feb  8 2021 Music
drwxr-xr-x 2 sysadmin sysadmin  6 Feb  8 2021 Pictures
drwxr-xr-x 2 sysadmin sysadmin  6 Feb  8 2021 Public
drwxr-xr-x 2 sysadmin sysadmin  6 Feb  8 2021 Templates
drwxr-xr-x 2 sysadmin sysadmin  6 Feb  8 2021 Videos
sysadmin@localhost:~$
```

3. Find past used commands using history

```
/usr/share/man/zh_TW/man5/passwd.5.gz
/usr/share/man/zh_TW/man8/chpasswd.8.gz
/var/lib/dpkg/info/base-passwd.list
/var/lib/dpkg/info/base-passwd.md5sums
/var/lib/dpkg/info/base-passwd.postinst
/var/lib/dpkg/info/base-passwd.postrm
/var/lib/dpkg/info/base-passwd.preinst
/var/lib/dpkg/info/base-passwd.templates
/var/lib/dpkg/info/passwd.conf files
/var/lib/dpkg/info/passwd.list
/var/lib/dpkg/info/passwd.md5sums
/var/lib/dpkg/info/passwd.postinst
/var/lib/dpkg/info/passwd.postrm
/var/lib/dpkg/info/passwd.preinst
/var/lib/dpkg/info/passwd.prerm
```

```
sysadmin@localhost:~$ history
```

- 1 man ls
- 2 ls -lh
- 3 clear
- 4 ls -lh
- 5*
- 6 locate passwd
- 7 history

```
sysadmin@localhost:~$ █
```